

1.INTERFACE :

Objective

To write a Java program to demonstrate the implementation of an interface using the implements keyword and access its methods and variables through a class.

2. Steps

1. Start the program.
 2. Create an interface named I1.
 3. Declare a variable x = 100 inside the interface.
 4. Declare two methods display() and show() in the interface.
 5. Create a class A that implements the I1 interface.
 6. Implement the display() method in class A.
 7. Implement the show() method in class A.
 8. Create an object obj of class A.
 9. Call the display() method using the object.
 10. Call the show() method using the object.
 11. Access and display the interface variable x.
 12. Display the results.
 13. Stop the program.
-

3. Algorithm

Algorithm: Interface Implementation

Step 1: Start.

Step 2: Create an interface I1.

Step 3: Declare variable x = 100 and methods display() and show() inside the interface.

Step 4: Create class A using the implements I1 keyword.

Step 5: Implement the display() and show() methods in class A.

Step 6: Create an object obj of class A.

Step 7: Call display() using the object.

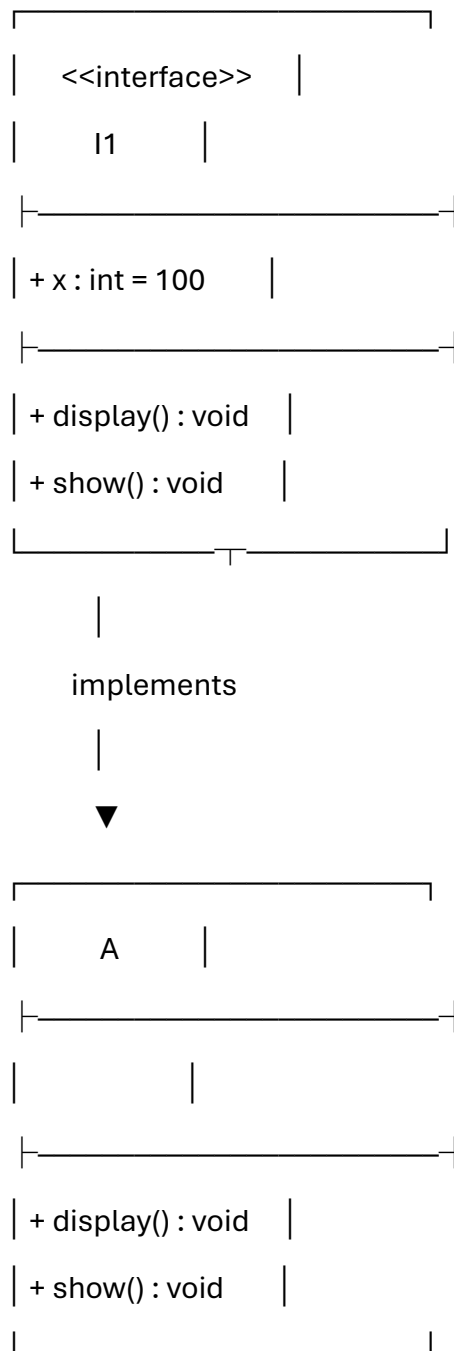
Step 8: Call show() using the object.

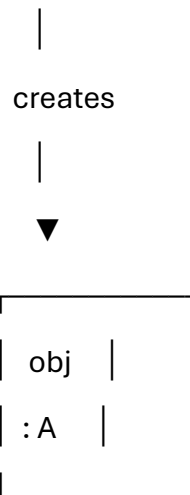
Step 9: Access the interface variable x using the object.

Step 10: Display all the results.

Step 11: Stop.

4. UML Diagram





Code :

```

interface I1{
    int x=100;
    void dispaly();
    void show();
}

class a implements I1{
    @Override
    public void dispaly(){
        System.out.println("it is from interface");
    }
    public void show(){
        System.out.println("method from interface");
    }
}

public class Interface {

    public static void main(String[] args) {
        a obj=new a();
  
```

```
obj.dispaly();

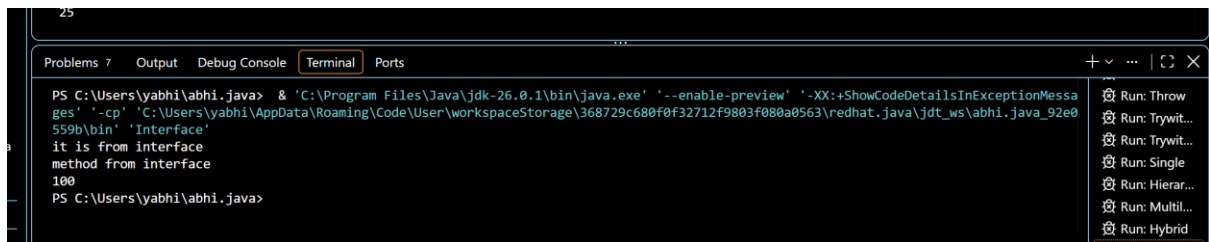
obj.show();;

System.out.println(obj.x);

}

}
```

Output :



The screenshot shows a terminal window in a Java IDE. The terminal output is as follows:

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'Interface'
it is from interface
method from interface
100
PS C:\Users\yabhi\abhi.java>
```

On the right side of the terminal window, there is a vertical list of run configurations with icons:

- Run: Throw
- Run: Trywit...
- Run: Trywit...
- Run: Single
- Run: Hierar...
- Run: Multil...
- Run: Hybrid

2. INTERFACE TO INTERFACE :

. Objective

To write a Java program to demonstrate inheritance between interfaces using the extends keyword and implement the derived interface using a class.

2. Steps

1. Start the program.
 2. Create an interface I1.
 3. Declare the variable x = 20 in interface I1.
 4. Declare the methods show() and display() in I1.
 5. Create another interface I2 that extends I1.
 6. Declare the variable y = 50 in interface I2.
 7. Declare the methods show() and play() in I2.
 8. Create class A that implements interface I2.
 9. Implement the show() method in class A.
 10. Implement the play() method in class A.
 11. Implement the display() method inherited from I1.
 12. Create an object obj of class A.
 13. Call display() using the object.
 14. Call show() using the object.
 15. Call play() using the object.
 16. Display the value of x.
 17. Display the results.
 18. Stop the program.
-

3. Algorithm

Algorithm: Interface-to-Interface Inheritance

Step 1: Start.

Step 2: Create interface I1 with variable x = 20 and methods show() and display().

Step 3: Create interface I2 using extends I1.

Step 4: Declare variable y = 50 and method play() in I2.

Step 5: Create class A using implements I2.

Step 6: Implement all the methods of I1 and I2 in class A.

Step 7: Create an object obj of class A.

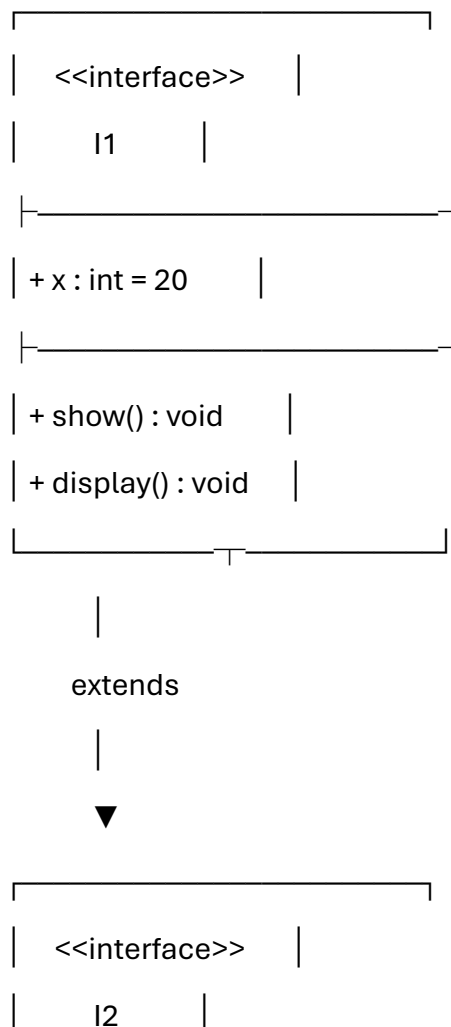
Step 8: Call display(), show(), and play() using obj.

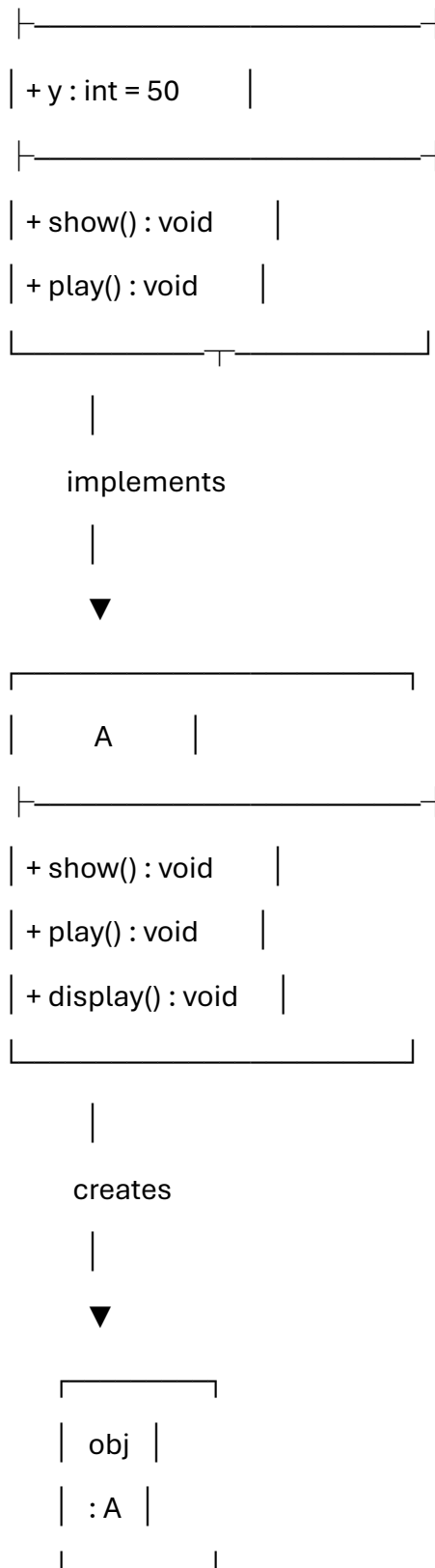
Step 9: Display the value of x.

Step 10: Display the output.

Step 11: Stop.

4. UML Diagram





Output :

```

implements from i1
it is implements from interface
y value from I250
20

```

3.C1 extends C2 :

Objective

To write a Java program to demonstrate multiple inheritance using multiple interfaces along with class inheritance.

Steps

1. Start the program.
2. Create interface I1 with method method1().
3. Create interface I2 with method method2().
4. Create interface I3 with method method3().
5. Create class C2 with the show() method.
6. Create class C1 that extends C2 and implements I1, I2, and I3.
7. Implement method1(), method2(), and method3() in C1.
8. Create an object obj of class C1.
9. Call show() from C2.
10. Call method1() from I1.
11. Call method2() from I2.
12. Call method3() from I3.
13. Display the results.
14. Stop the program.

Algorithm

Step 1: Start.

Step 2: Define interfaces I1, I2, and I3 with their respective methods.

Step 3: Define class C2 with the show() method.

Step 4: Define class C1 that extends C2 and implements I1, I2, and I3.

Step 5: Implement all three interface methods in C1.

Step 6: Create an object of C1.

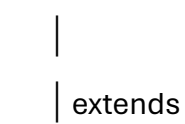
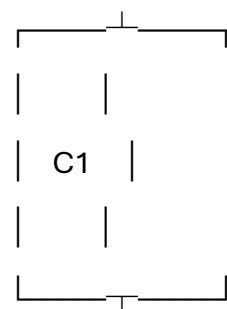
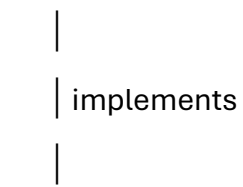
Step 7: Use the object to call show(), method1(), method2(), and method3().

Step 8: Display the output.

Step 9: Stop.

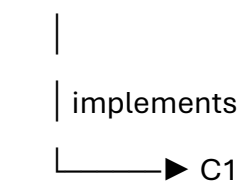
UML Diagram

<<interface>> I1

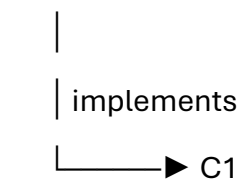


C2

<<interface>> I2



<<interface>> I3



Output :

```
This is C2
```

```
I1 method
```

```
I2 method
```

```
I3 method
```

Multiple :

Objective

To write a Java program to demonstrate multiple inheritance using multiple interfaces and access their variables and methods through a implementing class.

2. Steps

1. Start the program.
 2. Create interface I1.
 3. Declare variable x = 20 and method display() in I1.
 4. Create interface I2.
 5. Declare variable y = 50 and method play() in I2.
 6. Create interface I3.
 7. Declare variable z = 30 and method show() in I3.
 8. Create class A that implements I1, I2, and I3.
 9. Implement the display() method of I1.
 10. Implement the play() method of I2.
 11. Implement the show() method of I3.
 12. Create an object obj of class A.
 13. Call the display() method.
 14. Call the play() method.
 15. Call the show() method.
 16. Display the value of y.
 17. Display the results.
 18. Stop the program.
-

3. Algorithm

Algorithm: Multiple Inheritance Using Interfaces

Step 1: Start.

Step 2: Define interface I1 with variable x = 20 and method display().

Step 3: Define interface I2 with variable y = 50 and method play().

Step 4: Define interface I3 with variable z = 30 and method show().

Step 5: Create class A that implements all three interfaces.

Step 6: Implement the methods display(), play(), and show() in class A.

Step 7: Create an object obj of class A.

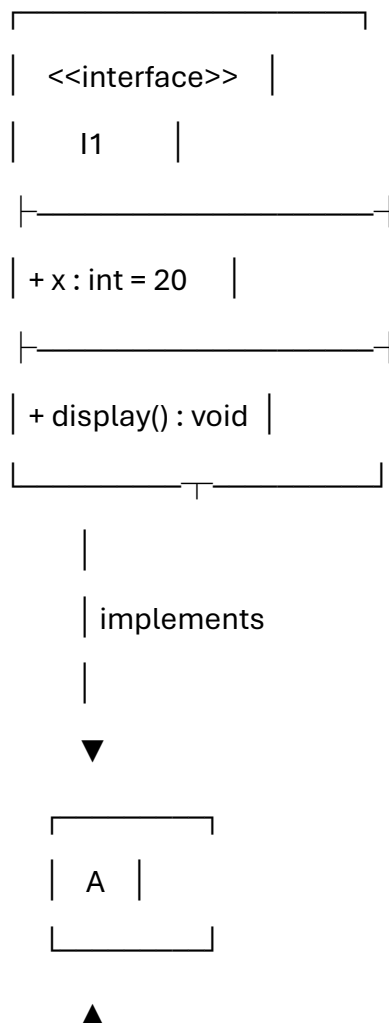
Step 8: Call display(), play(), and show() using the object.

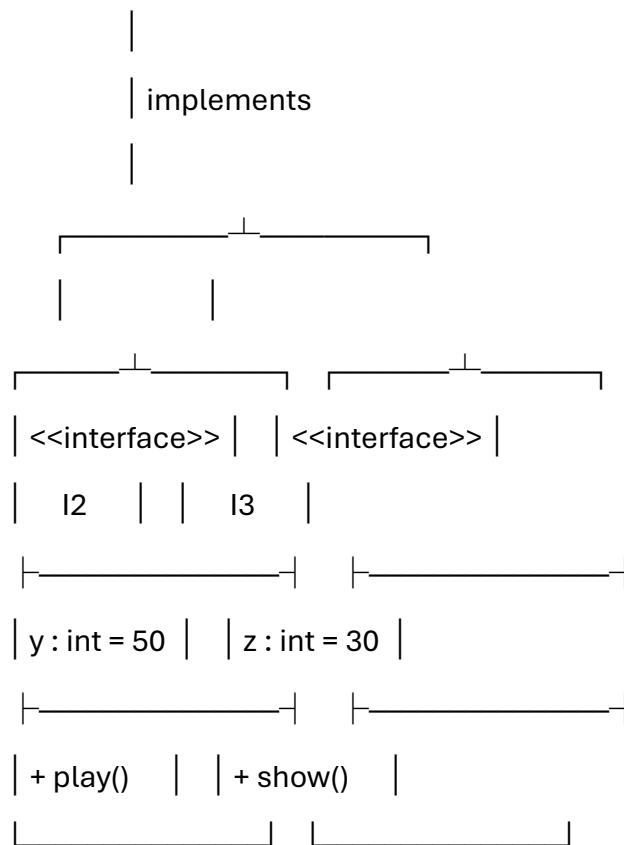
Step 9: Access and display the variable y.

Step 10: Display the output.

Step 11: Stop.

4. UML Diagram





Code :

```

interface I1{
    int x=20;
    void display();
}
interface I2{
    int y=50;
    void play();
}
interface I3{
    int z=30;
    void show();
}
  
```

```

class A implements I1,I2,I3{
    public void display(){
        System.out.println(x);
    }
    public void play(){
        System.out.println("it is from interface 2");
    }
    public void show(){
        System.out.println(z);
    }
}

```

```

public class Multiple {
    public static void main(String[] args) {
        A obj=new A();
        obj.display();
        obj.play();
        obj.show();
        System.out.println(obj.y);
    }
}

```

Output :

```

20
it is from interface 2
30
50

```